arXiv:2408.07089v1 [cs.LG] 9 Aug 2024

InfinityMATH : A Scalable Instruction Tuning Dataset in Programmatic Mathematical Reasoning

Bo-Wen Zhang*
bwzhang@baai.ac.cn
Beijing Academy of Artificial Intelligence
Beijing, China

Yan Yan*
yanyanustb@126.com
China University of Mining & Technology Beijing
Beijing, China

Lin Li
lilin000105@163.com
China University of Mining & Technology Beijing
Beijing, China

Guang Liu
liuguang@baai.ac.cn
Beijing Academy of Artificial Intelligence
Beijing, China

Abstract

Recent advancements in Chain-of-Thoughts (CoT) and Program-of-Thoughts (PoT) methods have greatly enhanced language models' mathematical reasoning capabilities, facilitating their integration into instruction tuning datasets with LLMs. However, existing methods for large-scale dataset creation require substantial seed data and high computational costs for data synthesis, posing significant challenges for scalability. We introduce **InfinityMATH** , a scalable instruction tuning dataset for programmatic mathematical reasoning. The construction pipeline emphasizes decoupling numbers from mathematical problems to synthesize number-independent programs, enabling efficient and flexible scaling while minimizing dependency on specific numerical values. Fine-tuning experiments with open-source language and code models, such as Llama2 and CodeLlama, demonstrate the practical benefits of InfinityMATH . These fine-tuned models, showed significant relative improvements on both in-domain and out-of-domain benchmarks, ranging from 184.7% to 514.3% on average. Additionally, these models exhibited high robustness on the *GSM8K+* and *MATH+* benchmarks, which are enhanced version of test sets with simply the number variations. InfinityMATH ensures that models are more versatile and effective across a broader range of mathematical problems. The data is available at https://huggingface.co/datasets/flagopen/InfinityMATH.

CCS Concepts

• **Computing methodologies** → **Language resources**; *Natural language generation.*

Keywords

programmatic mathematical reasoning, data augmentation, data synthesis, decoupled numeric dependencies, logical inconsistencies

*The corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '24, October 21–25, 2024, Boise, ID, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0436-9/24/10
https://doi.org/10.1145/3627673.3679122

ACM Reference Format:
Bo-Wen Zhang, Yan Yan, Lin Li, and Guang Liu. 2024. InfinityMATH : A Scalable Instruction Tuning Dataset in Programmatic Mathematical Reasoning. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24), October 21–25, 2024, Boise, ID, USA.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3627673.3679122

1 Introduction

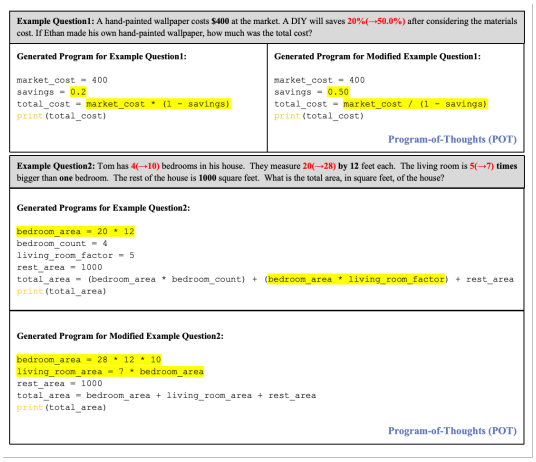

Figure 1: Examples of *Logical Inconsistencies in Reasoning* in LLM-generated programs when simply numerical variations

Mathematical reasoning involves understanding concepts, making logical deductions, and performing complex calculations, which are essential for evaluating the overall abilities of Large Language Models (LLMs) [11]. Enhancing model performance in mathematical reasoning is a hot research area. Several studies indicate that fine-tuning on math-specific datasets, covering problems from basic arithmetic to advanced algebra and geometry, significantly improves performance[2].

Recent research highlights that CoT [24] and PoT [4] techniques enhance mathematical reasoning capabilities through guiding models to sequentially unfold reasoning steps, or integrating executable program statements, allowing complex computations to be handled by a program interpreter. Therefore, several studies generate reasoning processes or programming solutions for mathematical

problems using models like GPT-4 to synthesize instruction tuning datasets, like OpenMathInstruct-1 [21], MetaMathQA [25] and MathInstruct [26]. However, the limited availability of high-quality mathematical reasoning problems and the high computational cost for data synthesis restrict large-scale data production.

In the context of these scaling challenges, we observed *Logical Inconsistencies in Reasoning* in LLM-generated programs. Figure 1 shows that minor numerical variations in problems lead to unexpected changes in the program's calculation logic, resulting in reasoning errors. For instance, in **Example Question 1**, altering a discount from 20% to 50% results in an illogical switch from multiplication to division, contradicting consistent discount principles. Similarly **Example Question 2** highlights a spatial reasoning error where changing the bedroom size results in an incorrect adjustment to the calculation logic. These examples show that slight numerical changes disrupt the program's logic, revealing issues with the reasoning robustness to numerical variations.

We propose **InfinityMath** , a scalable instruction tuning dataset for programmatic mathematical reasoning. The construction of InfinityMath involves a multi-step pipeline: numerical values in mathematical problems are firstly identified and abstracted to generate "universal templates" for questions, then an LLM (such as GPT-4) generates programs independent of these specific values, and finally, the templates are repopulated with varied numbers to broaden the dataset while preserving reasoning logic. Consequently, InfinityMath comprises 101,380 extensible data points generated from 7 high-quality mathematical datasets, formulating an "**infinite mathematical instruction tuning dataset**".

We fine-tuned the 7B versions of the Llama2 and Aquila2 (language models), as well as the Codellama (code foundation model), with the InfinityMath dataset and evaluated on four in-domain and five out-of-domain benchmarks. Empirical results indicate these fine-tuned models consistently outperform other state-of-the-art models of comparable size on the most benchmarks. Furthermore, to investigate the *Logical Inconsistencies in Reasoning* phenomenon, we created enhanced versions of the GSM8K and MATH test sets, designated GSM8K+ and MATH+, which only modified the numerical values within the problems. Comparative analysis reveals that models trained on InfinityMath exhibit superior accuracy and robustness compared to those trained on other PoT datasets.

2 Related Work

The use of large models for solving mathematical problems has become a key research focus, serving as a crucial indicator for assessing the performance of LLMs in complex multi-hop and quantitative reasoning tasks. Researchers have explored various methods to enhance the mathematical reasoning capabilities of LLMs, bridging the gap between closed-source and open-source models.

The Chain-of-Thought (CoT) method, introduced by Wei et al.[24], decomposes mathematical problems into smaller, interconnected tasks. This step-by-step approach enables models to solve complex issues incrementally. Wang et al.[23] enhanced this with the Self-Consistency method, where the model generates multiple reasoning processes and selects the most likely correct answer through voting. Li et al. [13] further developed this by transforming a single prompt into multiple ones, checking intermediate reasoning steps, and applying weighted voting. However, CoT requires language models to generate and compute mathematical expressions, which can be inefficient for tasks like solving polynomial equations or calculus problems.

Chen et al.[4] introduced Program-of-Thoughts (PoT), which delegates computational steps to an external interpreter (e.g., Python), while maintaining natural language reasoning. Luo et al.[15], using code-specific Evol-Instruct, fine-tuned StarCoder [12] to create WizardCoder, outperforming major LLMs on code generation benchmarks. Bi et al.[3] proposed the Code and Instruction Reasoning Score (CIRS), evaluating the correlation between code and reasoning abilities. Their experiments on datasets like AsDiv[16] and GSM8K [6] highlighted PoT's superiority over CoT, especially in handling complex computations with external libraries like SymPy.

Fine-tuning methods leverage the generic features learned by large models to perform well on new tasks with minimal training. Yue et al.[26] introduced MathInstruct, a dataset mixing CoT and PoT principles, fine-tuned on Llama22[22] and codellama [19]. Yu et al.[25] proposed MetaMathQA, based on GSM8K and MATH, incorporating more reasoning paths and fine-tuned on LLaMA-2. Toshniwal et al.[21] introduced OpenMathInstruct-1, synthesizing solutions from GSM8K and MATH's code interpreters, fine-tuned on Mistral [9], Llama-2, and CodeLlama.

3 Methodology

The motivation for constructing InfinityMath is to tackle the challenge of scaling data while addressing logical inconsistencies in reasoning. We propose a simple yet efficient pipeline that decouples numerical values from mathematical problems and synthesizes a large number of similar problems without significantly increasing computational costs. The goal is to overcome the dependencies of problem synthesis on specific numerical values, thereby maximizing data utilization and enhancing the robustness of models.

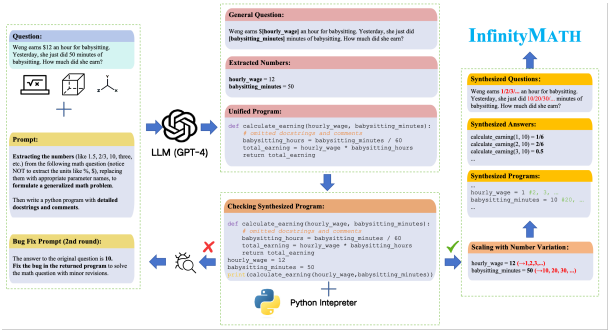

Figure 2: The construction pipeline for InfinityMath

3.1 Data Synthesis

Drawing from algebraic thinking, we hypothesize that each mathematical problem can be transformed into a more general, number-independent "generic problem" and "generic solution". Therefore, when using LLMs (such as GPT-4 [1]) for data synthesis, we diverge

Table 1: Statistics of InfinityMath , the number of questions in source datasets and successful generated samples.

Dataset	AQuA-RAT [14]	GSM8K [6]	MATH [8]	NUMGLUE [17]	MathQA [2]	TheoremQA [5]	Deepmind-Mathematics [20]	Total
Samples	53822	7254	5021	14228	20319	310	426	101380
No. of Question	97467	7473	7500	20359	29837	800	490	-
Success Rate	55.23%	97.07%	66.95%	69.89%	68.12%	38.75%	86.94%	-

from the classical PoT method by not directly generating solution code. Instead, we employ a multi-step process:

First, numerical constants in problems are masked to create "generic problems", with these masked parts replaced by appropriate variable names as placeholders. This generalization simplifies program synthesis and prepares for scalability. Subsequently, LLMs are utilized to generate programs that solve these generic problems. Specifically, the generation of function call-based programs instead of inline code is required. This approach enhances the reusability of the synthesized code and supports efficient scaling. Moreover, including a docstring in the function code improves readability by describing the function's purpose, the variables used, and the expected output. This establishes a clear relationship between the solution and the variables. Comments within the program explain the computational logic of each line, incorporating CoT rationale-like content. Therefore, the synthesized programs must include docstrings and comments to ensure clarity and comprehensibility.

To minimize LLM usage and reduce computational costs, we designed and compared several versions of prompt templates. Ultimately, we developed a multi-task prompt that requires the LLM to complete the tasks described above simultaneously, as illustrated in Figure 2. Moreover, we included an example to leverage the in-context learning capability of the LLM, enhancing the model's accuracy. Additionally, to facilitate post-processing, we strictly defined the format of the returned answers, thereby increasing the usability of the synthesized data.

Afterward, using simple rules, the program solution to the original math problem can be synthesized directly from the **"General Question"**, **"Extracted Numbers"** and **"Unified Program"** obtained in the 1st-round response. The accuracy of the data synthesis process is verified by executing the synthesized code with a Python interpreter and comparing the results to the ground truth answer. For more complex mathematical problems, the generated programs often contain bugs. Instead of repeatedly adjusting hyperparameters inefficiently, we leverage the multi-turn dialogue capability of LLMs. By providing feedback from the checking program, we construct a **Bug Fix Prompt** for the 2nd round, supplying the LLM with the correct answer to the original problem and requesting minimal modifications to the erroneous code, as shown in Figure 2.

To enhance the model's generalization capability in mathematical reasoning, we selected seven high-quality open-source math datasets and used the aforementioned method to generate solutions. The statistics of InfinityMath (each problem corresponding to one data point) is shown in Table 1.

3.2 Scaling with Data Augmentation

Recent data augmentation studies have focused on using LLMs to rewrite problems or generate different solutions for the same problem. However, there has been limited research on approaches that do not rely on additional LLM usage. To address this gap, we conducted the following work.

During data synthesis, we generated generic mathematical problems and function call-based programs. As shown in Figure **??**, new problems and solution code can be generated by reversing the process: replacing variable placeholders with numbers.

Suppose the original mathematical problem uses k numbers, which are replaced with variable placeholders during data synthesis. Since we can choose whether to replace variables with numbers or to retain the variable placeholder, there are $2^k - 1$ replacement options for each group of reasonable variable-to-number mappings, resulting in $2^k - 1$ possible program solutions.

When modifying the program, we remove variable assignments and docstring parts related to the replaced variables, then replace occurrences of the variables with the original numbers. We verify correctness by checking if the modified code runs correctly and produces the expected output, ensuring the generated programs are logically correct.

It is crucial that replacement numbers follow certain rules to maintain the intended meaning (e.g., ensuring the number of people is an integer). Therefore, we require the LLM to provide reasonable number ranges or criteria during synthesis to ensure the validity of the numbers.

4 Experiments

4.1 Experimental Setup

We selected the open-source models CodeLlama, Llama2, and Aquila2, each with 7B parameters. These models were fine-tuned using InfinityMath and other datasets to validate instruction tuning. CodeLlama and Llama2 were aligned to an Alpaca-like instruction structure, while Aquila2 followed the Aquila-v2 structure. We used a learning rate of 2×10^{-5} for CodeLlama and Llama2, and 1×10^{-5} for Aquila2, with a global batch size of 128 for all models.

We evaluated the effectiveness of our data on five in-domain test sets: GSM8K, MATH, AQuA-RAT, NumGLUE, Mathematics, and four out-of-domain test sets: SVAMP [18], SimulEq [10], SAT-Math [27], MMLU-Math [7]. For datasets containing both CoT and PoT, we used the Mammoth [26] evaluation framework, which involves first evaluating with a PoT prompt. If the generated program fails, a second evaluation is done using a CoT prompt to potentially improve results. For datasets with only PoT prompts (including InfinityMath), we exclusively used PoT prompts for evaluation with no retries. All evaluations were performed in a **0-shot setting** without additional examples.

4.2 Experimental Results

4.2.1 Main Results. The evaluation results in Table 2 show that the InfinityMath dataset consistently enhances performance across different base models compared to other datasets.

Table 2: The overall evaluation results on both in-domain and out-of-domain benchmarks.

Base Model	Training Dataset	In-domain Evaluation					Out-of-domain Evaluation			
		GSM8K	MATH	AQuA	Mathematics	NumGLUE	SVAMP	SimulEq	SAT-Math	MMLU-Math
Llama2	-	14.60	2.50	30.30	6.20	29.90	34.50	4.60	22.70	30.60
	WizardMath	54.90	10.70	26.30	9.30	36.10	36.10	12.80	25.40	31.10
	MetaMathQA	**66.50**	19.80	-	-	-	-	-	-	-
	MathInstruct	53.60	**31.50**	**44.50**	46.30	61.20	67.70	41.20	**42.70**	**42.60**
	MathInstruct(pot)	47.23	24.44	15.75	41.00	63.82	59.40	39.88	10.91	20.53
	InfinityMath	60.80	29.40	38.58	**63.70**	**75.91**	**76.10**	**56.03**	25.91	30.70
		(+316.44%)	(+1076.00%)	(+27.30%)	(+927.42%)	(+153.88%)	(+120.58%)	(+1118.09%)	(+14.13%)	(+0.33%)
CodeLlama	-	16.30	10.92	18.50	28.60	26.68	50.60	3.89	15.91	20.77
	MathInstruct	59.40	33.40	**47.20**	55.40	66.40	71.40	45.90	**40.50**	**48.30**
	MathInstruct(pot)	56.86	29.88	16.54	61.70	62.19	69.60	40.27	11.82	17.66
	InfinityMath	**65.80**	**34.06**	38.18	**67.10**	**71.40**	**77.00**	**74.71**	32.73	37.27
		(+303.07%)	(+211.91%)	(+106.38%)	(+134.65%)	(+167.62%)	(+52.17%)	(+1820.05%)	(+105.76%)	(+79.47%)
Aquila2	-	20.47	3.78	14.96	4.80	13.82	23.30	6.23	19.09	12.63
	MathInstruct	50.19	25.98	15.35	**71.79**	61.30	44.40	40.08	14.55	17.86
	MathInstruct(pot)	49.66	**26.50**	12.60	47.40	**72.36**	50.10	46.11	17.27	**26.80**
	InfinityMath	**51.25**	26.18	**23.62**	42.20	68.04	**66.00**	**59.92**	**19.55**	21.77
		(+150.40%)	(+592.59%)	(+57.88%)	(+779.17%)	(+392.12%)	(+183.26%)	(+861.97%)	(+213.82%)	(+72.41%)

For in-domain evaluation, InfinityMath demonstrates substantial improvements. For example, with InfinityMath , Llama2 achieves a 316.44% improvement on GSM8K, 1076.00% on MATH, and 927.42% on Mathematics. Similar gains are observed with CodeLlama and Aquila2. In out-of-domain evaluation, InfinityMath continues to show significant performance gains. With InfinityMath , Llama2 achieves a 120.58% improvement on SVAMP and 1118.09% on SimulEq. Notable improvements are also seen with CodeLlama and Aquila2.

While InfinityMath does not perform as well as MathInstruct on some benchmarks due to the inclusion of CoT data and the advantage of two evaluation opportunities, it still consistently enhances performance across all evaluated benchmarks. This highlights the effectiveness and robustness of the InfinityMath dataset in improving the mathematical reasoning capabilities of LLMs.

4.2.2 Effect of Scaling. We used subsets of data from GSM8K and MATH within InfinityMath to compare the effectiveness of scaling methods. By replacing the variables in the InfinityMath subsets with different numbers, we fine-tuned CodeLlama and Llama2 models. To ensure that the improvements were due to the scaling method and not merely the increased data volume, we compared checkpoints based on the same number of training steps rather than epochs. The results, shown in Table 3, demonstrate that augmenting the GSM8K and MATH subsets of InfinityMath with different numbers increased data volume and improved accuracy. This validates the effectiveness of our synthetic dataset method.

Table 3: The results of fine-tuning using data augmentation with InfinityMath subsets on GSM8K and MATH (G+M).

BaseModel	InfinityMath -G+M	GSM8K	MATH
Llama2	w/o scaling	51.78	22.78
	w/ scaling	55.42 (+7.03%)	23.98 (+5.27%)
CodeLlama	w/o scaling	62.02	30.86
	w/ scaling	63.38 (+2.19%)	31.32 (+1.49%)

4.2.3 Results of GSM8K+ and MATH+. To investigate logical inconsistencies in reasoning within LLM-generated programs, we constructed the GSM8K+ [1] and MATH+ [2] evaluation datasets. By replacing two sets of numbers in the original datasets and conducting rigorous manual checks, we created 1319×3 and 3818×3 evaluation data points, respectively. The evaluation results are shown in Table 4, where x and y represent the number of instances where the model generated at least one correct answer and all three correct answers for the same problem, respectively. The y/x ratio indicates the preservation of logical consistency in reasoning.

[1]Available at https://huggingface.co/datasets/flagopen/
[2]Available at https://huggingface.co/datasets/flagopen/MATHplus

Table 4: The comparison results on GSM8K+ and MATH+.

Base Model	Training Dataset	GSM8K+			MATH+		
		x	y	%	x	y	%
Llama2	MathInstruct(pot)	827	463	56.0	1484	581	39.1
	InfinityMath	**974**	**605**	**62.1**	**1741**	**684**	**39.3**
Codellama	MathInstruct(pot)	913	568	62.2	1720	696	40.5
	InfinityMath	**1022**	**672**	**65.8**	**1985**	**850**	**42.8**

4.2.4 DocString Ablation. To analyze the impact of CoT rationale-like descriptions, we conducted an ablation study focusing on docstrings. By removing docstrings in InfinityMath , we performed comparative experiments using CodeLlama on four in-domain evaluation sets. The experimental results, shown in Table 5, indicate that even though docstrings do not directly affect program execution, they enhance the model's reasoning capabilities.

Table 5: Impact of docstrings on the 7B codellama model.

	AQuA	GSM8K	MATH	NUMGLUE
w/o docstring (%)	31.5	58.7	28.2	69.9
w/ docstring (%)	39.4	62.3	31.1	68.0

5 Conclusion

We open-source InfinityMath , a mathematical reasoning instruction tuning dataset with each data point including a generic problem and a solution template, allowing for easy scaling into an infinite dataset. The fine-tuning experiments show that InfinityMath effectively alleviates logical inconsistencies in reasoning.

Acknowledgements

This work was supported by National Key R&D Program of China (2022ZD0116312).

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Aida Amini, Saadia Gabriel, Peter Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. 2019. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. *arXiv preprint arXiv:1905.13319* (2019).

[3] Zhen Bi, Ningyu Zhang, Yinuo Jiang, Shumin Deng, Guozhou Zheng, and Huajun Chen. 2024. When Do Program-of-Thought Works for Reasoning?. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 17691–17699.

[4] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. 2022. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588* (2022).

[5] Wenhu Chen, Ming Yin, Max Ku, Pan Lu, Yixin Wan, Xueguang Ma, Jianyu Xu, Xinyi Wang, and Tony Xia. 2023. Theoremqa: A theorem-driven question answering dataset. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.

[6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).

[7] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).

[8] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874* (2021).

[9] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).

[10] Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. 2016. MAWPS: A math word problem repository. In *Proceedings of the 2016 conference of the north american chapter of the association for computational linguistics: human language technologies*. 1152–1157.

[11] Guillaume Lample and François Charton. 2019. Deep Learning for Symbolic Mathematics. *arXiv preprint arXiv:1912.01412* (2019).

[12] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).

[13] Yifei Li, Zeqi Lin, Shizhuo Zhang, Qiang Fu, Bei Chen, Jian-Guang Lou, and Weizhu Chen. 2022. Making large language models better reasoners with step-aware verifier. *arXiv preprint arXiv:2206.02336* (2022).

[14] Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. 2017. Program induction by rationale generation: Learning to solve and explain algebraic word problems. *arXiv preprint arXiv:1705.04146* (2017).

[15] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568* (2023).

[16] Shen-Yun Miao, Chao-Chun Liang, and Keh-Yih Su. 2021. A diverse corpus for evaluating and developing English math word problem solvers. *arXiv preprint arXiv:2106.15772* (2021).

[17] Swaroop Mishra, Arindam Mitra, Neeraj Varshney, Bhavdeep Sachdeva, Peter Clark, Chitta Baral, and Ashwin Kalyan. 2022. NumGLUE: A suite of fundamental yet challenging mathematical reasoning tasks. *arXiv preprint arXiv:2204.05660* (2022).

[18] Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. Are NLP models really able to solve simple math word problems? *arXiv preprint arXiv:2103.07191* (2021).

[19] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[20] David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. 2019. Analysing mathematical reasoning abilities of neural models. *arXiv preprint arXiv:1904.01557* (2019).

[21] Shubham Toshniwal, Ivan Moshkov, Sean Narenthiran, Daria Gitman, Fei Jia, and Igor Gitman. 2024. OpenMathInstruct-1: A 1.8 Million Math Instruction Tuning Dataset. *arXiv preprint arXiv:2402.10176* (2024).

[22] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[23] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).

[24] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[25] Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2023. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284* (2023).

[26] Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. 2023. Mammoth: Building math generalist models through hybrid instruction tuning. *arXiv preprint arXiv:2309.05653* (2023).

[27] Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. 2023. Agieval: A human-centric benchmark for evaluating foundation models. *arXiv preprint arXiv:2304.06364* (2023).